JX Runtime + Onyx Engine

Onyx runs the model. JX guards the door.

Onyx Engine serves one model per process. JX Runtime installs it, spawns it, health-checks it and puts scoped keys, TLS, pairing and an audit log in front of it.

JX Runtime does

- Access keys & pairing
- TLS & network posture
- Model installs & fit checks
- Crash detection & restart
- Metrics without content
- Audit log

Onyx Engine does

- Inference on llama.cpp
- Continuous batching
- Streaming over SSE
- Tool calls & grammars
- Embeddings
- Multimodal input

Switch JX Runtime to Onyx Engine

01 Install

```
jx-runtime backend install --engine onyx
```

A prebuilt onyx-engine, checked against its release checksums, lands in the data directory.

02 Select

```
execution.backend: "onyxengine"
```

One setting. Its adapter subclasses the llama.cpp one: same spawn, streaming and restart code.

03 Serve

```
one onyx-engine per loaded model
```

Bound to 127.0.0.1, polled at /health, its OpenAI endpoints proxied through.

04 Share

```
Authorization: Bearer jxr_…
```

Apps and paired devices reach it through JX Runtime's keys, TLS and rate limits.

`github.com/jxburros/JX-Runtime` `github.com/JX-Holdings-LLC/Onyx-Engine`

Both MIT licensed. JX Runtime is alpha; its release binaries are not yet code-signed.